

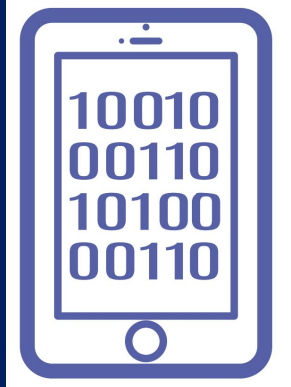
MOBİL UYGULAMA



İSTANBUL DENEYAP TEKNOLOJİ ATÖLYESİ

Ders Saatleri : 09.00 - 13.00 - Lise Sabah Grubu (Sınıfta) - (9 Temmuz- 16 Temmuz Hafta İçi)

Ders Saatleri : 13.30 - 17.30 - Orta Öğle Grubu (Sınıfta) - (9 Temmuz- 16 Temmuz Hafta İçi)



MOBİL UYGULAMA DERSİ

2.Ders:Mobil Uygulamalar-Flutter-Dart



T3 DENEYAP EĞİTMENİ (Hazırlayan ve Sunan): Muhammed Emin Korkunç

(Bilgiyar Mühendisi)

Bu sunumun kopyalanması, paylaşılması ve yazılı izinsiz kullanımı KESİNLİKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunc-100ba2215 | github.com/muhammedkorkunc

MOBİL UYGULAMA

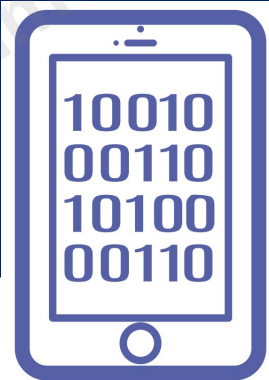


Sabah Grubu

- 09.00 - 09.50: 1. Ders
- 09.50 - 10.00: 1. Mola
- 10.00 - 10.50: 2. Ders
- 10.50 - 11.00: 2. Mola
- 11.00 - 11.50: 3. Ders
- 11.50 - 12.00: 3. Mola
- 12.00 - 13.00: 4. Ders

Öğle Grubu

- 13.30 - 14.20: 1. Ders
- 14.20 - 14.30: 1. Mola
- 14.30 - 15.20: 2. Ders
- 15.20 - 15.30: 2. Mola
- 15.30 - 16.20: 3. Ders
- 16.20 - 16.30: 3. Mola
- 16.30 - 17.30: 4. Ders



1- TERMİNAL

KURULUM VİDEOSU=https://www.youtube.com/watch?v=zdj7Np_qzGw

TERMİNAL (flutter_console.bat dosyası içinde)

KOD= flutter doctor –android-licenses

KOD= flutter doctor

KOD= flutter create app

KOD= cd app

KOD= flutter run

 Tek yapmanız gereken, terminalde oluşturduğumuz app1 klasörünü tutup VSCode ekranının üzerine sürükleyip bırakmak!

 Sol taraftaki Explorer (Dosya Gezgini) butonuna tıkladığımızda projenin tüm dalları önümüze serilecek. Bizim için en en önemli dosya hangisi biliyor musunuz? Tabii ki lib klasörünün altındaki main.dart dosyası! Tüm iş bu dosyada dönüyor."

2- WEB DE DART DİLİ ÇALIŞTIRMAK

Dart Kod Çalıştırma Sitesi = <https://dartpad.dev>

Muhammed Emin Korkunç

1.1. Gözle: Temel Kavramlar

- **Dart Nedir?** Google tarafından geliştirilen, çeşitli platformlarda (iOS, Android vb.) çalışabilen istemci uygulamaları üretmeyi amaçlayan bir programlama dilidir.
- **Flutter ile İlişkisi:** * Flutter uygulamaları Dart programlama dili kullanılarak kodlanır.
- **Geliştirici Dostu Yaklaşım:** * Dart, diğer dillerle benzerlikler taşıdığı için kolay öğrenilir.
 - Başlangıçta zorlayıcı gelebilecek bazı kuralları, aslında kullanıcıların karşılaşabileceği kritik hataları daha **geliştirme aşamasında** çözmek için tasarlanmıştır.

4- Uygulama Çakılmaları ve "Sessiz Hatalar"

Sessiz Hatalar (Silent Errors):

- Değer atanmamış değişkenlerin kullanılması gibi basit nedenler uygulamaların aniden çakılmasına (kapanmasına) yol açabilir.
- Bu çakılmalarda kullanıcıya hiçbir hata mesajı gösterilmez.

Sonuçları:

- Kullanıcı sorunun ne olduğunu anlayamaz ve geliştiriciye hata bildirimi yapamaz.
- Bu durum, kullanıcıların uygulamaya olan güvenini ve memnuniyetini doğrudan olumsuz etkiler.
- Dart dili, aldığı katı önlemlerle bu tür çökmeleri daha kod yazılırken engeller.

5- Dart Dilinin Temel Özellikleri - 1

Nesne Yönelimli (OOP) Bir Dildir:

- Dart'ta değişkenlere atanan her ifade (sayılar, fonksiyonlar dahil) bir sınıftan türetilen birer **nesnedir**.

Erişim Belirleyiciler (Erişim Kontrolü):

- Java'daki gibi `public` veya `protected` anahtar kelimeleri kullanılmaz.
- Bir ifadenin kütüphaneye özgü (gizli) kalması isteniyorsa, isminin önüne **alt çizgi** (`_`) eklenir.
- *Örnek:* `_ozelDegiskenim`

6- Dart Dilinin Temel Özellikleri - 2

- **Sıkı Veri Tipi Kontrolü (Strongly Typed):**
 - Değişkenler yalnızca tek bir türde (int, String vb.) veri tutabilir.
 - Uygunsuz bir değer ataması yapıldığında, Dart derleyicisi hatayı anında yakalar ve kodun derlenmesine izin vermez.

Zayıf Tipli Dillerdeki Risk (Örnek Senaryo):

- fiyat = 100 (Sayı) ve vergi = 18 (Sayı) olsun.
- JavaScript veya PHP gibi dillerde vergi değişkenine yanlışlıkla "kdv" (Yazı) atanabilir.
- fiyat + vergi işlemi yapıldığında sonuç matematiksel bir toplam yerine "100kdv" olur.
- Bu hatalı veri bir ödeme sistemine gönderilirse sistem kilitlenir.
- **Dart, sıkı veri tipi kontrolü ile bu tarz hataları tamamen önler.**

Tip Çıkarımı Mekanizması (Type Inference):

- Değişken tanımlarken her zaman belirli bir veri tipi belirtmek zorunlu değildir.
- Dart, değişkene atanan ilk değere bakarak türünü kendisi tahmin edebilir.
- Değişkenler var anahtar sözcüğü ile tanımlanır.
- *Örnek:* `var sayi = 10;` yazıldığında Dart bunun bir `int` olduğunu anlar. Geliştiriciler bu mekanizmayı kullanmayı önerir.

8- Dart Dilinin Temel Özellikleri - 4

Null Güvenliği (Null Safety):

- Değer atanmamış (boş) değişkenlerin kullanılmasından doğan `null exception` hatalarını engeller.
- Programcı özellikle izin vermediği sürece Dart değişkenleri asla `null` değeri alamaz.

Sıcak Yükleme (Hot Reload):

- Yazılan kodların, o sırada cihazda çalışmakta olan uygulamaya anında enjekte edilmesini sağlar.
- Uygulamayı kapatıp açmaya gerek kalmadan, çalışır vaziyetteyken anlık düzenlemeler yapılmasına imkan tanır.

9- 1.2. Gözle ve Uygula: Dart Uygulamaları

Temel Yapı: İlk inceleyeceğimiz temel Dart uygulaması, matematiksel işlemler, fonksiyon tanımlamaları ve değişken yönetimini bir arada barındırmaktadır.

Çekirdek Kütüphane (**dart:core**):

- Temel veri tipleri, listeler ve standart işlemler için gerekli araçları sağlar.
- Her Dart uygulamasına **otomatik olarak** eklenir (Ekstra bir kod yazmaya gerek yoktur).

Özel Kütüphaneler: Matematiksel işlemler, asenkron yapılar veya veri tabanı işlemleri gibi özel görevler için ilgili kütüphanelerin dışarıdan projeye dahil edilmesi gerekir.

10- Kütüphane Dahil Etme (import)

import Anahtar Sözcüğü: Dış kütüphaneleri uygulamaya eklemek için kullanılır.

dart:math Kütüphanesi:

- Kodumuzun 1. satırında yer alan import "dart:math"; ifadesiyle projeye dahil edilmiştir.
- Bu kütüphane; üs alma (pow), kök alma, trigonometrik hesaplamalar ile pi ve e gibi matematiksel sabitleri uygulamamıza kazandırır.

11- Giriş Noktası: `main()` Fonksiyonu

Uygulamanın Başlangıcı: Dart uygulamaları her zaman `main()` adı verilen üst düzey (top-level) fonksiyondan başlar ve buradan dallanır.

Geri Dönüş Tipleri ve `void`:

- Sıkı veri tipi kontrolü nedeniyle, fonksiyonların hangi türde değer döndüreceği belirtilmelidir.
- Eğer bir fonksiyon geriye değer döndürmüyorsa `void` olarak tanımlanır.
- *Örnek:* `main()` fonksiyonu değer döndürmediği için `void` `main()` şeklinde tanımlanmıştır (Satır 2).

Değişken Tanımlama Yolları: Dart'ta değişkenler açıkça tipi yazılarak (String, num vb.) veya **var** anahtar sözcüğüyle tanımlanabilir.

Tip Çıkarımı (Type Inference):

- var kullanıldığında Dart, değişkene atanan ilk değere bakarak türünü otomatik belirler.
- *Örnek:* Kodun 3. satırındaki `var sayi = 40;` ifadesinde Dart, `sayi` değişkeninin bir tam sayı (`int`) olduğuna otomatik karar verir.

Fonksiyon Yapısı: Fonksiyonlar isimleri ile çağrılır ve dışarıdan veri almak için parametre kullanırlar.

Veri Aktarımı (Satır 4 & 6):

- `karesiniAl` fonksiyonu `sayi` adında bir parametre beklemektedir.
- `main` içinde `karesiniAl(sayi)` çağrısı yapıldığında, `sayi` değişkenindeki **40** değeri fonksiyona aktarılır.

14- Değer Döndürme (return) ve Konsol Çıktısı

Hesaplama ve **return** (Satır 6 & 7):

- **karesiniAl** fonksiyonu geriye sayısal bir değer döndüreceği için **num** tipiyle tanımlanmıştır.
- **dart:math** içindeki **pow(sayim, 2)** fonksiyonu ile sayının karesi hesaplanır.
- **return** anahtar kelimesi hesaplanan sonucu alır ve fonksiyonun çağrıldığı 4. satıra geri fırlatır.

Ekrana Yazdırma (print**):** Fonksiyondan dönen sonuç doğrudan **print()** fonksiyonuna beslenir ve konsola yazdırılır.

```
import "dart:math";

void main() {
  var sayi = 40;
  print(karesiniAl(sayi));
}

num karesiniAl(sayim) {
  return (pow(sayim, 2));
}
```

```
1 import "dart:math";
2 void main() {
3   var sayi = 40;
4   print(karesiniAl(sayi));
5 }
6
7 num karesiniAl(sayim) {
8   return (pow(sayim, 2));
9 }
10
```

16- Yorum Satırları ve Alanları

Yorum Yapıları Nedir? Programcıların kendilerine veya birlikte çalıştıkları ekip arkadaşlarına kod içerisinde notlar/mesajlar bırakmasını sağlayan yapılardır.

Derleyici Davranışı: Dart derleyicisi bu alanları tamamen görmezden gelir; yorumlar program kodunun çalışmasını etkilemez.

Kullanım Amaçları:

- Bir değişkenin veya fonksiyonun görevini açıklamak.
- Test veya hata ayıklama esnasında belirli kod satırlarının geçici olarak çalıştırılmasını engellemek.

17- Yorum Türleri: Satır ve Alan Yorumları

1. Tek Satırlık Yorumlar (//):

- Kısa ve tek satıra sığacak açıklamalar için satırın başına // işareti eklenir.
- Örnek: // Bu bir tek satırlık yorumdur.

```
void main() {  
    // Dart uygulamaları main()  
    // fonksiyonu ile başlar.  
    // Bu uygulama konsola  
    "uygulama başladı" yazmaktadır.  
    print("uygulama başladı");  
}
```

2. Çok Satırlık Yorum Alanları (/* ... */):

- Tek satırı aşan, bir fonksiyonun işlevini veya parametrelerini detaylıca anlatan uzun açıklamalar için kullanılır.
- /* ile başlar ve */ işareti görülene kadar aradaki tüm karakterler yorum kabul edilir.

```
void main() {  
    /*  
    Dart uygulamaları main() fonksiyonu ile  
    başlar.  
    Bu fonksiyon konsola "uygulama  
    başladı" yazmaktadır  
    */  
    print("uygulama başladı");  
}
```

18- İşleçler (Operatörler)

İşleç Nedir? Programlama dilinin derleyicisine belirli bir matematiksel, mantıksal veya ilişkisel işlemi gerçekleştirme komutu veren sembollerdir.

İfade (Expression): Değerlerin bir işleç ile birleştirilmesiyle oluşur (Örnek: $4 + 6$). Derleyici bu ifadeleri otomatik olarak işleyip sonuç üretir.

İşleç Türleri:

- **Ön Ek İşleçler:** İfadenin önüne gelir (Ör: $-sayı$ ile sayının işaretini tersine çevirmek).
- **Son Ek İşleçler:** İfadenin arkasına eklenir (Ör: $ifade++$ veya bir nesne özelliğine erişirken kullanılan $.$ işleci).

19- İşleç Önceliği (Operator Precedence)

Öncelik Sırası: Bir ifade içinde birden fazla işleç varsa hangisinin önce çalışacağını belirler.

Hiyerarşi: Tablodaki üst grupta yer alan işleçler, altındakilere göre daha yüksek önceliğe sahiptir.

Çarpma/Bölme vs. Toplama/Çıkarma:

- Aynı ifade içinde çarpma ve toplama yan yanaysa, önce çarpma işlemi yapılır.
- Örnek:** $3 * 2 + 4$ ifadesinde önce $3 * 2 = 6$ hesaplanır, ardından $6 + 4$ yapılarak sonuç **10** bulunur (Sıralı gidilip 12 bulunmaz).

Mantıksal İşleçler: Koşulları birleştirmek için **VE (&&)** ile **VEYA (||)** işleçleri kullanılır. (Ör: $\text{sayi} > 0 \ \&\& \ \text{sayi} < 10$)

```
void main() {  
    print(4 + 6); // 10  
    print(3 - 6); // -3  
  
    var sayi = 10;  
    print(-sayi); // -10 (Sadece ekrana basarken  
işaret değişir, değişken hala 10dur)  
  
    print(3 * sayi); // 30  
    print(sayi / 3); // 3.3333333333333335  
    print(sayi ~/ 3); // 3 (Tam kısım alınır)  
    print(sayi % 3); // 1 (Kalan bulunuz)  
}
```

20- Aritmetik İşleçler

Dart dilinde standart matematiksel işlemler için kullanılan temel operatörler şunlardır:

- **+** : Toplama
- **-** : Çıkarma (veya Ön ek olarak İşaret Değiştirme)
- ***** : Çarpma
- **/** : Bölme (Double/Ondalıklı sonuç üretir)
- **~/** : Tam Sayı Bölmesi (Bölümün sadece tam kısmını alır)
- **%** : Mod Alma (Bölme işleminden kalan sayıyı verir)

```
void main() {
  print(4 + 6); // 10
  print(3 - 6); // -3

  var sayi = 10;
  print(-sayi); // -10 (Sadece ekrana basarken
işaret değişir, değişken hala 10dur)

  print(3 * sayi); // 30
  print(sayi / 3); // 3.3333333333333335
  print(sayi ~/ 3); // 3 (Tam kısım alınır)
  print(sayi % 3); // 1 (Kalan bulunuz)
}
```

21- Kod İncelemesi: İşaret Değiştirme ve Bölme Detayları

Ön Ek Eksi (-) Tuzağı: `print(-sayi);` ifadesi ekrana negatif değer basar ancak **değişkenin içindeki değeri kalıcı olarak değiştirmez.**

- Değerin kalıcı değişmesi için atama yapılmalıdır: `sayi = -sayi;`

Normal Bölme (/) vs Tam Bölme (~/):

- `10 / 3` işlemi yapıldığında sonuç ondalıklı olarak **3.333...** çıkar.
- `10 ~/ 3` işlemi yapıldığında tilda (~) sembolü sayesinde ondalık kısım atılır ve doğrudan tam sayı olan **3** sonucu elde edilir.

Mod Alma (%): `10 % 3` ifadesi, 10'un 3'e bölümünden kalan **1** değerini üretir.

```
void main() {  
    print(4 + 6); // 10  
    print(3 - 6); // -3  
  
    var sayi = 10;  
    print(-sayi); // -10 (Sadece ekrana basarken  
işaret değişir, değişken hala 10dur)  
  
    print(3 * sayi); // 30  
    print(sayi / 3); // 3.3333333333333335  
    print(sayi ~/ 3); // 3 (Tam kısım alınır)  
    print(sayi % 3); // 1 (Kalan bulunuz)  
}
```

22- Ön Ek ve Son Ek Arttırma-Azaltma İşleçleri

Ön Ek İşleçler (**++sayi**, **--sayi**):

- İşlemin etkisi, satırdaki kod (örneğin ekrana yazdırma) yürütülmeden **önce** gerçekleşir.
- Örnek: `print(++sayi);` ifadesi, önce sayıyı 11 yapar, sonra ekrana 11 basar.

Son Ek İşleçler (**sayi++**, **sayi--**):

- İşlemin etkisi, satırdaki kod yürütüldükten **sonra** değişkene yansır.
- Örnek: `print(sayi++);` ifadesi, ekrana sayının mevcut değerini (11) basar; ancak bu satır geçildikten sonra sayının içindeki değer 12 olur.

```
void main() {  
    var sayi = 10;  
    print(++sayi); // 11 -> Önce arttırdı, sonra yazdırdı.  
    print(sayi++); // 11 -> Önce yazdırdı, hafızada 12 yaptı.  
    print(sayi); // 12 -> Güncel değer ekranda.  
    print(sayi--); // 12 -> Önce yazdırdı, hafızada 11 yaptı.  
    print(--sayi); // 10 -> Önce azalttı (11'den 10'a), sonra yazdırdı.  
    print(sayi); // 10 -> Güncel değer.  
}
```

23- İlişkileri Kontrol Etme İşleçleri

Tanım: Verilen iki değer birbirine göre durumunu (eşitlik, büyüklük, küçüklük) kıyaslamak için kullanılır.

Geri Dönüş Tipi: Bu ifadelerin sonucu her zaman **Boolean** (**true** veya **false**) türündedir.

Temel İlişkisel İşleçler:

- **==** : Eşit mi?
- **!=** : Eşit değil mi? (Farklı mı?)
- **>** / **<** : Büyük mü? / Küçük mü?
- **>=** / **<=** : Büyük eşit mi? / Küçük eşit mi?

```
void main() {  
    print(2 == 2); // true  
    print(2 != 2); // false  
    print(2 > 2); // false
```

```
    var sayi = 2; // Not: Orijinal metindeki eksik tanımlama  
    düzeltildi  
    print(sayi >= 2); // true  
    print(3 <= sayi); // false  
}
```

24- Atama ve Birleşik Atama İşleçleri

Temel Atama (=): Bir değeri bir değişkene aktarmak için kullanılır. (`sayi = 2;`)

Birleşik Atama (Kısayol) İşleçleri: Aritmetik işlem ile atama işlemini tek adımda birleştirir.

- `sayi += 3` —> Açılımı: `sayi = sayi + 3`
- `sayi *= 3` —> Açılımı: `sayi = sayi * 3`
- `sayi ~/ 4` —> Açılımı: `sayi = sayi ~/ 4` (Tamsayı bölme ataması)

Kritik Hata (Null Pointer / TypeError):

- Tanımlanmış fakat **ilk değer ataması yapılmamış (null)** bir değişken üzerinde birleşik atama (`+=`, `*=`) yapılmaya çalışılırsa program çöker.
- Dart konsolda şu hatayı fırlatır: `TypeError: null has no properties`

```
void main() {  
  var sayi = 2; // Not: İlk değer ataması  
  zorunludur  
  sayi += 3; // sayi = sayi + 3;  
  print(sayi); // 5  
  
  sayi *= 3; // sayi = sayi * 3;  
  print(sayi); // 15  
  
  sayi ~/ 4; // sayi = sayi ~/ 4; -> 15 ~/ 4  
  print(sayi); // 3  
}
```

Güvenli Kodlama Kuralı: Bu tür çökmelerden kaçınmak için tanımlanan her değişkene mutlaka bir **ilk değer (initial value)** atanmalıdır.

25- Durumsal Atama (Ternary Operator)

Tanım: `if-else` yapısının tek satırda yazılmasını sağlayan durumsal bir atama kısayoludur.

Sözdizimi (Syntax): `Koşul ? "Doğruysa Alınacak Değer" : "Yanlışsa Alınacak Değer";`

Sembollerin Anlamı:

- **? (Soru İşareti):** Kendinden önceki koşulu sorgular (`if` bloğu gibi).
- **: (İki Nokta Üst Üste):** Koşul sağlanmadığında devreye girer (`else` bloğu gibi).

Örnek: `var sonuc = (sayi % 2 == 0) ? "çift" : "tek";`

26- Durumsal Atama vs If-Else Yapısı

Avantajı: Süslü parantez `{ }` kalabalığını önler, kodu oldukça kısaltır ve okunabilirliği artırır.

Sınırları / Dezavantajı:

- Bu yapı **yalnızca atama işlemlerinde** kullanılabilir.
- Eğer şartın doğru veya yanlış olması durumunda sadece değer atamak yerine, arka arkaya farklı kodlar/işlemler çalıştırılacaksa geleneksel **if-else** yapısı kullanılmak zorundadır.

```
void main() {  
    var sayi = 3;  
  
    // Ternary Kullanımı  
    var sonuc = (sayi % 2 == 0) ? "çift" : "tek";  
    print(sonuc); // tek  
  
    /* Yukarıdaki işlemin geleneksel if-else ile  
    yapılışı:  
    var sonuc;  
    if(sayi % 2 == 0) {  
        sonuc = "çift";  
    } else {  
        sonuc = "tek";  
    }  
    */  
}
```

27- Mantıksal İşleçler

Boolean (**true/false**) türündeki mantıksal değerler veya koşullar üzerinde mantıksal kombinasyonlar kurmak için kullanılır.

1. Mantıksal DEĞİL (!): Verilen Boolean değer tersini (zıttını) alır.

- **Örnek:** **!true** ifadesi **false** sonucunu üretir.

2. Mantıksal VE (&&):

- Bağlanan tüm koşulların **doğru olması** durumunda **true** döndürür. Koşullardan en az biri yanlışsa sonuç **false** olur.
- **Örnek:** **true && false** → **false**

3. Mantıksal VEYA (||):

- Bağlanan koşullardan **en az birinin doğru olması** **true** dönmesi için yeterlidir. Sadece tüm koşullar yanlışsa **false** döner.
- **Örnek:** **true || false** → **true**

```
void main() {  
    print(!true);      // false -> Doğrunun  
                        tersi yanlıştır.  
    print(true && true); // true -> VE  
                        işleminde ikisi de doğruysa sonuç  
                        doğrudur.  
    print(true && false); // false -> VE  
                        işleminde tek bir yanlış sonucu yanlış  
                        yapar.  
    print(true || false); // true -> VEYA  
                        işleminde tek bir doğru sonuç için  
                        yeterlidir.  
}
```

Rezerve Kelimeler: Her programlama dilinde olduğu gibi Dart dilinde de sistem tarafından özel amaçlarla ayrılmış (rezerve edilmiş) kelimeler vardır.

Kısıtlama: Bu anahtar sözcükler kod yazarken kesinlikle **değişken, fonksiyon veya sınıf ismi** (tanımlayıcı) olarak kullanılamaz.

Örnekler: `void`, `main`, `var`, `final`, `const`, `return`, `if`, `else` gibi kelimeler Dart dilinin yapı taşlarıdır ve isim olarak seçilemezler.

29- Değişken Kavramı ve Bellek İlişkisi

Değişken Nedir? Uygulamaların cihaz belleğinde (RAM) geçici olarak tuttuğu verilere ve değerlere referans veren (onları işaret eden) isimlendirilmiş kayıtlardır.

Çalışma Mantığı: * `var sayi = 3;` yazıldığında, `sayi` kelimesi bellekteki 3 değerini temsil eden bir referanstır.

- Kod içerisinde `sayi` ifadesini kullandığınızda, derleyici bu ifadeyi arka planda referans verdiği 3 değeriyle değiştirerek işleme sokar.

Veri Güncelleme: Değişkenlerin içindeki değerler, atama işlemleri kullanılarak program akışı boyunca istenildiği kadar güncellenebilir.

```
void main() {  
    var sayi = 3;  
    print(++sayi); // 4 -> Bellekteki 3  
    değerine referans veren sayi  
    değişkeni 1 arttırıldı.  
}
```

30- Değişken Tanımlama Yaklaşımları ve Tip Çıkarımı

Tanımlama Yolları: `var sayi1 = 10;` (Tür belirtmeden, otomatik)

- `int sayi2 = 10;` (Tür belirterek, manuel)

Tip Çıkarımı (Type Inference):

- Dart, değişkene atanan ilk değere bakarak onun hangi veri tipinde (`int`, `String`, `double` vb.) olması gerektiğine kendisi karar verir.

Öneri: Dart geliştiricileri ve resmi dokümantasyonlar, kodun daha temiz kalması adına veri türünün açıkça yazılması yerine **`var`** anahtar sözcüğünün tercih edilmesini önermektedir.

```
void main() {  
    var sayi = 3;  
    print(++sayi); // 4 -> Bellekteki  
    3 değerine referans veren sayi  
    değişkeni 1 arttırıldı.  
}
```

31- Değiştirilemez Yapılar: final Anahtar Sözcüğü

Tanımlanan bir değişkenin programın ilerleyen aşamalarında **tekrar değiştirilmesini engellemek** istiyorsak **final** kelimesi kullanılır.

Özelliği:

- Salt okunur (read-only) bir değişken oluşturur.
- Değer yalnızca bir kez atanabilir (Run-time / Çalışma zamanı sabiti).

Hata Durumu: `final isim = "Dart";` tanımından sonra `isim = "Programlama";` şeklinde yeni bir değer atanmaya çalışılırsa derleyici hata verir:

"The final variable 'isim' can only be set once."

```
void main() {  
    // final kullanımı  
    final isim = "Dart";  
    // isim = "Programlama"; // HATA! Bu satırın yorumu kaldırılırsa "The final variable 'isim' can only be set once" hatası alınır.  
  
    // const kullanımı  
    const versiyon = 2.12;  
    // versiyon = 3; // HATA! Bu satırın yorumu kaldırılırsa "Constant variables can't be assigned a value" hatası alınır.  
}
```

32- Derleme Zamanı Sabitleri: const Anahtar Sözcüğü

Tanım: Tamamen sabit, hiçbir şekilde değişmeyecek **derleme süreci sabitlerini** (Compile-time constants) tanımlamak için kullanılır.

final ile Farkı: `const` ile tanımlanan değişkenlerin değeri program daha derlenirken (çalıştırılmadan önce) kesin olarak bilinmelidir ve sabittir.

Hata Durumu: `const versiyon = 2.12;` tanımından sonra `versiyon = 3;` yazılırsa sistem çöker ve şu hatayı verir:

“Constant variables can't be assigned a value.”

```
void main() {  
    // final kullanımı  
    final isim = "Dart";  
    // isim = "Programlama"; // HATA! Bu satırın yorumu kaldırılırsa "The final variable 'isim' can only be set once" hatası alınır.  
  
    // const kullanımı  
    const versiyon = 2.12;  
    // versiyon = 3; // HATA! Bu satırın yorumu kaldırılırsa "Constant variables can't be assigned a value" hatası alınır.  
}
```

33- Öntanımlı Veri Tipleri - Sayılar (int)

Dart dilinde iki temel sayısal veri tipi bulunur: int ve double.

Tam sayıları ifade etmek için int veri tipi kullanılır.

int veri tipinin sınırları, uygulamanın çalıştığı platforma göre değişiklik gösterir:

- Mobil Cihazlarda (64-bit): -2^{53} ile $2^{53} - 1$ arasındadır.
- Web Uygulamalarında (JavaScript sınırları): -2^{53} ile $2^{53} - 1$ arasındadır.

Soru / Cevap

Soru: Tam sayı sınırlarında negatif değerler -2^{53} 'e kadar uzanırken, pozitif değerler neden $2^{53} - 1$ kadardır? Neden pozitif tarafta bir sayı eksiktir?

Cevap: Bilgisayar sistemlerinde "0" (sıfır) sayısı pozitif bölgesinde kabul edilerek indekslenir. Toplam sembol alanının yarısı negatif sayılara, diğer yarısı ise sıfır ve pozitif sayılara ayrıldığı için pozitiflerin üst sınırı 1 eksik olur.

Dart'ta sayılar farklı sayı sistemlerinde veya üstel (bilimsel) gösterimle tanımlanabilir.

```
void main() {  
    var sayi = 1;           // 10'luk sistemde doğrudan tanımlama  
    var onaltılık = 0xA00; // 16'lık (Hexadecimal) sistem (Onluk karşılığı: 2560)  
    var ustel = 8e5;        // Üstel gösterim ( $8 * 10^5 = 800.000$ )  
}
```

35- Öntanımlı Veri Tipleri - Sayılar (double)

- Kesirli ve ondalıklı değerlerin tutulması için **double** veri tipi kullanılır.
- Ondalıklı sayılar doğrudan yazılabileceği gibi üstel gösterimle de tanımlanabilir.

```
void main() {  
    var ondalikli = 1.1; // Doğrudan ondalıklı tanımlama  
    var ustel2 = 1.42e5; //  $1.42 * 10^5$  anlamına gelir (142.000)  
}
```

Muhammed Emin Korkunç

36- Öntanımlı Veri Tipleri - Metinler (String)

Dart'ta metinsel ifadeler String nesneleridir ve UTF-16 standardında karakter katarlarıdır.

String metin oluşturmak için tek tırnak (' . . . ') veya çift tırnak (" . . . ") kullanılabilir. İkisi arasında işlevsel bir fark yoktur.

Kaçış Karakteri (\): Metin içinde kullanılan tırnak işaretlerinin hata vermemesi için önüne ters bölü (\) işareti konur.

```
void main() {  
  var s1 = 'tek tırnak içinde metin.';  
  var s2 = "çift tırnak içinde metin.";  
  
  // Tek tırnak içinde tek tırnak kullanmak için kaçış (\) karakteri:  
  var s3 = 'Tek tırnak içinde kesme (\) işaretini göstermek.';  
  
  // Çift tırnak içinde tek tırnak doğrudan kullanılabilir:  
  var s4 = "Çift tırnak içinde kesme (') işaretini göstermek";  
}
```

Dart, String ifadelerin içine dinamik olarak değişken enjekte etmeyi destekler.

Bunun için String içinde değişken adının önüne \$ karakteri getirilir.

Metinleri birleştirmek için geleneksel + işleci de kullanılabilir.

```
void main() {  
  var s = 'ekleyebilir';  
  // String interpolation kullanımı:  
  print('Dart metinler içine farklı metinleri $s');  
  
  // + işleci ile birleştirme alternatifi:  
  // print('Dart metinler içine farklı metinleri ' + 'ekleyebilir');  
}
```

Mantıksal (koşul) verilerin tutulması için **bool** veri tipi kullanılır.

`bool` tipindeki değişkenler yalnızca iki değer alabilir: **true (doğru)** veya **false (yanlış)**.

```
void main() {  
    var tekMi = true;  
    var ciftMi = false;  
}
```

Programlamada en çok kullanılan yapıların başında gelen grupta elemanlarıdır.

Listeler benzer özellikteki nesnelerin (sayılar, metinler vb.) bir arada tutulmasını sağlar.

Dart, listenin içerdiği elemanlara bakarak türünü otomatik olarak tahmin eder (Tür Çıkarımı / Type Inference).

int türünde eleman içeren bir listeye String türünde bir eleman eklenmeye çalışılırsa derleme hatası oluşur.

Liste elemanları sıfırdan başlayan sıra numaraları (indeks) ile çağrılır.

Listenin tümünü String formatında çıktı almak için `toString()` metodu kullanılır.

```
void main() {  
    var sayilar = [1, 2, 4];  
    var alisverisListem = [  
        "ekmek",  
        "süt",  
        "yoğurt", // Son elemandan sonraki virgül hata üretmez, okunurluğu artırır.  
    ];  
  
    print(alisverisListem[0]); // Çıktı: ekmek (0. indeks ilk elemandır)  
    print(alisverisListem.toString()); // Listenin tamamını metin olarak basar.  
}
```

41-Liste Elemanlarını Güncelleme ve Hata Durumu

Listedeki mevcut elemanlar atama işleci (=) kullanılarak güncellenebilir.

Sınırlar dahilinde olmayan, yani tanımlı bulunmayan bir indekse doğrudan atama yapılamaz; bu durum çalışma zamanı hatasına sebep olur.

```
void main() {  
    var alisverisListem = ["ekmek", "süt", "yoğurt"];  
    var sayilar = [1, 2, 4];  
  
    alisverisListem[1] = "meyve suyu";    // "süt" yerine "meyve suyu" atanır.  
    sayilar[2] = 5;    // 4 değeri 5 olarak güncellenir.  
  
    // Hatalı Kullanım:  
    // sayilar[3] = 10; // HATA! 3 numaralı indeks henüz mevcut değil.  
}
```

42- Listeye Eleman Ekleme ve Silme Metotları

add(eleman): Listenin sonuna yeni bir eleman ekler.

removeAt(indeks): Belirtilen indeksteki elemanı siler. Silme işleminden sonra gelen elemanların indeks numaraları otomatik olarak birer azalır.

insert(indeks, eleman): Listede araya eleman eklemek için kullanılır. Belirtilen indekse elemanı yerleştirir, sonrakileri kaydırır.

```
void main() {  
    var sayilar = [1, 2, 5];  
    var alisverisListem = ["ekmek", "meyve suyu", "yoğurt"];  
  
    sayilar.add(10);        // Listenin sonuna 10 ekler -> [1, 2, 5, 10]  
    alisverisListem.removeAt(2);    // 2. indeksteki "yoğurt" silinir.  
    alisverisListem.insert(2, "zeytin");    // 2. indekse "zeytin" eklenir.  
  
    print(alisverisListem.toString());  
}
```

43- Listelerde Döngü Kullanımı - forEach

Listelerdeki tüm elemanlar üzerinde sırayla işlem yapmak için **forEach** metodu kullanılabilir.

Bu metot, her bir elemanı kendisine parametre olarak verilen **anonim fonksiyona (arrow function)** gönderir.

```
void main() {  
    var alisverisListem = ["ekmek", "meyve suyu", "zeytin"];  
  
    // Her elemanı konsolda ayrı bir satıra yazdırır:  
    alisverisListem.forEach((element) => print(element));  
}
```

44- Listelerde Arama Yapma - firstWhere

firstWhere: Listeyi verilen test koşuluna göre tarar ve koşula uyan **ilk elemanın kendisini** döndürür.

Koşul hiçbir eleman tarafından sağlanmazsa hata oluşmaması için **orElse** bloğu tanımlanabilir.

```
void main() {  
    var alisverisListem = ["ekmek", "meyve suyu", "zeytin"];  
  
    var sonuc = alisverisListem.firstWhere(  
        (element) => element == "gazete",  
        orElse: () {  
            return "aranan ifade bulunamadı";  
        },  
    );  
    print(sonuc); // Çıktı: aranan ifade bulunamadı  
}
```

45- Listelerde İndeks Arama - indexWhere

indexWhere: Test koşulunu sağlayan ilk elemanın **sıra numarasını (indeksini)** döndürür.

Eğer aranan eleman listede bulunamazsa metot geriye **-1** değerini çevirir.

```
void main() {  
    var alisverisListem = ["ekmek", "süt", "zeytin"];  
  
    var sonuc = alisverisListem.indexWhere((element) => element == "süt");  
  
    if (sonuc > -1) {  
        sonuc++; // İndeks 0'dan başladığı için kullanıcıya sunarken 1 artırıyoruz.  
        print("aranan ifade $sonuc. sırada");  
    } else {  
        print("aranan ifade bulunamadı");  
    }  
}
```

46- Listeleri Birleştirme - Yayılım İşleci (. . .)

İki listeyi tek bir listede birleştirmek için **Yayılım İşleci (Spread Operator - . . .)** kullanılır.

Bir listenin önüne . . . konulduğunda, o listenin tüm elemanları teker teker çözülerek diğer listenin içerisine aktarılır. döngü kurmaya kıyasla çok daha pratik bir yöntemdir.

```
void main() {  
    var dogalSayilar = [0, 1, 2, 4];  
    // dogalSayilar listesi tamSayilar listesinin içine eritiliyor:  
    var tamSayilar = [-1, -4, -11, ...dogalSayilar];  
  
    print(tamSayilar.toString());  
    // Çıktı: [-1, -4, -11, 0, 1, 2, 4]  
}
```

Yayılım işleci yerine alternatif olarak forEach döngüsü kullanılarak da listeler birleştirilebilir.

```
void main() {  
    var dogalSayilar = [1, 2, 3];  
    var tamSayilar = [-1, -2, -3];  
  
    // dogalSayilar'daki her eleman tek tek tamSayilar listesine ekleniyor:  
    dogalSayilar.forEach((sayi) => tamSayilar.add(sayi));  
  
    print(tamSayilar.toString());  
    // Çıktı: [-1, -2, -3, 1, 2, 3]  
}
```

48-Kümeler (Sets)

Kümeler, birbirinden farklı elementlerin tutulduğu **sirasız (unordered)** bir listedir.

Listelerden en büyük farkı: Kümelerde **aynı ifade iki farklı eleman olarak yer alamaz**, her eleman eşsizdir.

Küme tanımlamak için süslü parantezler (**{ }**) kullanılır.

Boş bir küme oluşturulurken süslü parantezden önce veri tipi (örneğin **<String>**) mutlaka belirtilmelidir; aksi halde harita (Map) ile karıştırılabilir.

```
void main() {  
    // Elemanları olan bir küme tanımlama  
    var haftalci = {"salı", "çarşamba", "perşembe", "cuma"};  
  
    // Boş bir String kümesi tanımlama  
    var gunler = <String>{};  
}
```

49-Kümelere Eleman Ekleme (add ve addAll)

add(eleman): Kümeye tek bir eleman eklemek için kullanılır.

addAll(digerKume): Mevcut bir kümeye başka bir kümenin tüm elemanlarını topluca ekler.

Kümeye halihazırda var olan bir değer tekrar eklenmek istendiğinde Dart **hata vermez**, ancak değeri ikinci kez kümeye **dahil etmez**.

```
void main() {  
    var haftalci = {"salı", "çarşamba", "perşembe", "cuma"};  
    var haftaSonu = {"cumartesi", "pazar"};  
    var gunler = <String>{};  
  
    gunler.add("pazartesi");  
    gunler.addAll(haftalci);  
    gunler.addAll(haftaSonu);  
  
    // "pazartesi" zaten var, tekrar eklemeyi denerseniz küme değişmez:  
    gunler.add("pazartesi");  
  
    print(gunler.toString());  
    // Çıktıda "pazartesi" sadece 1 kez görünür.  
}
```

50- Küme Elemanlarına Erişim ve Eleman Sayısı

Kümeler sırasız olduğu için listelerdeki gibi `küme[0]` şeklinde indeksle erişim yapılamaz.

`elementAt(indeks)`: Belirtilen sıradaki elemana erişmek için kullanılır (Sıralama 0'dan başlar).

`first`: Kümenin ilk elemanını doğrudan getirir.

`length`: Kümede toplam kaç adet eleman bulunduğunu verir.

```
void main() {  
    var gunler = {"pazartesi", "salı", "çarşamba"};  
  
    var ilkGun = gunler.elementAt(0); // 0. indeksteki eleman  
    var ilkGunAlternatif = gunler.first; // İlk elemana erişim  
    var gunSayisi = gunler.length; // Eleman sayısı  
  
    print("İlk gün: $ilkGun");  
    print("günler kümesinde $gunSayisi adet gün var.");  
}
```

51- Kümelerde Arama ve Eleman Silme

forEach: Küme elemanlarını sırayla tarayarak döngüsel işlemler yapmayı sağlar.

lookup(eleman): Eleman kümede varsa **kendisini**, yoksa **null** değerini döndürür.

contains(eleman): Eleman kümede varsa **true**, yoksa **false** döndürür.

remove(eleman): Belirtilen elemanı siler. Silme başarılıysa **true**, eleman yoksa **false** döner.

```
void main() {  
    var gunler = {"pazartesi", "salı", "çarşamba"};  
  
    // Tüm elemanları yazdırma  
    gunler.forEach((element) => print(element));  
  
    // lookup ile arama ve remove ile silme  
    if (gunler.lookup("salı") != null) {  
        gunler.remove("salı");  
    }  
}
```

52- Koşullu Silme ve Filtreleme (removeWhere, clear, where)

removeWhere: Belirtilen test koşulunu (fonksiyonunu) sağlayan tüm elemanları topluca siler.

clear(): Kümenin içindeki tüm elemanları silerek kümeyi tamamen boşaltır.

where: Belirli bir koşula uyan elemanları bularak yeni bir küme (Iterable) olarak geri döndürür.

```
void main() {  
    var sayilar = {1, 2, 3, 6, 10, 12};  
  
    var ondanKucukSayilar = sayilar.where((element) => element < 10);  
    print("10'dan küçükler: $ondanKucukSayilar");  
  
    // 9'dan büyük elemanları silme  
    sayilar.removeWhere((element) => element > 9);  
    print("9'dan büyükler silindikten sonra: $sayilar");  
  
    // Küme içeriğini tamamen boşaltma  
    sayilar.clear();  
}
```

53-Haritalar (Maps)

Haritalar, bir **anahtar (key)** ve buna bağlı bir **değerden (value)** oluşan nesneler kümesidir.

En önemli kural: Haritalarda **anahtarlar (keys)** **eşsiz (unique)** olmak zorundadır.

Bir değer (value) birden fazla farklı anahtara atanabilir.

Anahtar ve değerler farklı veri türlerinde olabilir (Örn: **int** anahtara karşılık **String** değer).

```
void main() {  
    // Tip çıkarımı ile harita oluşturmak  
    var elementler = {  
        1: "hidrojen",  
        3: "lityum",  
    };  
  
    // Belirli tiplerde boş harita (Map) tanımlamak  
    var baskentler = Map<String, String>();  
    var soygazlar = Map<int, String>();  
}
```

54- Haritalara Eleman Ekleme ve Değer Sorgulama

Haritaya eleman eklemek veya mevcut elemanı güncellemek için **harita[anahtar] = deger** yapısı kullanılır.

Köşeli parantez içinde sorgulanan anahtar haritada bulunuyorsa değeri döner, bulunmuyorsa **null** döner.

addAll(digerHarita): Aynı veri tipindeki iki haritayı birleştirmek için kullanılır. Farklı tipler birleştirilemez (hata verir).

```
void main() {  
    var elementler = {1: "hidrojen", 3: "lityum"};  
    var baskentler = Map<String, String>();  
  
    // Eleman ekleme  
    elementler[4] = "berilyum";  
    baskentler["tr"] = "Ankara";  
  
    var soygazlar = {2: "helyum", 10: "neon"};  
  
    // Haritaları birleştirme  
    elementler.addAll(soygazlar);  
    print(elementler);  
}
```

55-Harita Metotları (length, forEach, remove, update)

length: Haritadaki toplam anahtar-değer çifti sayısını verir.

forEach: Haritadaki tüm elemanları (**anahtar**, **deger**) parametreleriyle tarar.

remove(anahtar): Belirtilen anahtara ait çifti haritadan siler.

update(anahtar, (deger) => yeniDeger) veya doğrudan atama ile eleman güncellenebilir.

```
void main() {  
    var elementler = {2: "helyum", 10: "neon", 18: "argon"};  
  
    print("Eleman sayısı: ${elementler.length}");  
  
    // Tüm haritayı tarama  
    elementler.forEach((an, deg) => print("Atom No: $an, Element: $deg"));  
  
    // Eleman silme  
    elementler.remove(2); // 2 anahtarlı eleman silinir.  
  
    // Eleman güncelleme alternatifleri  
    elementler[10] = "NEON";  
    elementler.update(18, (value) => "ARGON");  
}
```

56- Haritalarda keys ve values Özellikleri

keys: Haritadaki tüm anahtarları getirir.

values: Haritadaki tüm değerleri getirir.

Bu iki özellik geriye **Iterable** tipinde bir yapı döndürür. **Iterable** listelerin kendine özgü metot ve işlevleri mevcuttur.

```
void main() {  
    var elementler = {"H": "helyum", "Li": "lityum"};  
  
    var simgeler = elementler.keys; // Anahtarlar kümesi (H, Li)  
    var isimler = elementler.values; // Değerler kümesi (helyum, lityum)  
  
    print(simgeler); // Çıktı: (H, Li)  
    print(isimler); // Çıktı: (helyum, lityum)  
}
```

57- Akış Yöneticileri - if-else Yapısı

Program kodlarının belirli bir şarta bağlı olarak çalıştırılmasını sağlar.

if ifadesine yazılan şart doğru (**true**) ise ilk blok (**if** bloğu) çalışır.

Şart sağlanmıyorsa (**false** ise) ikinci blok (**else** bloğu) devreye girer.

if bloğunun tanımlanması zorunludur ancak **else** bloğu isteğe bağlıdır.

```
void main() {  
    var sayi = 3;  
  
    // Çift/Tek kontrolü (%2 modu 0 ise çifttir)  
    if (sayi % 2 == 0) {  
        print("$sayi sayısı çifttir");  
    } else {  
        print("$sayi sayısı tektir");  
    }  
}
```

58- Çoklu Koşul Bağlama (else if)

Bir koşulun sağlanmaması durumunda **else if** ifadesi kullanılarak ardışık yeni koşullar sınanabilir.

Koşullardan herhangi biri sağlandığı an ilgili blok çalışır ve diğer bloklar atlanır.

```
void main() {  
    var mA = 80, mB = 60, mC = 40;  
  
    if (mA == 90 || mB == 90 || mC == 90) {  
        print("dik üçgen");  
    } else if (mA == mB && mB == mC) {  
        print("eşkenar üçgen");  
    } else if (mA == mB || mB == mC || mA == mC) {  
        print("ikizkenar üçgen");  
    } else {  
        print("özel bir üçgen değil");  
    }  
}
```

59- Problem Çözme Etkinliği - 1 (Çözümlü)

Soru: Verilen kodu, üçgenin iç açılarının toplamının 180 derece olmasını kontrol edecek şekilde geliştiriniz.

```
void main() {  
    var mA = 80;  
    var mB = 60;  
    var mC = 40;  
  
    // Öncelikle iç açılar toplamı 180 mi kontrol ediyoruz  
    if ((mA + mB + mC) == 180) {  
        if (mA == 90 || mB == 90 || mC == 90) {  
            print("Geçerli bir dik üçgen");  
        } else if (mA == mB && mB == mC) {  
            print("Geçerli bir eşkenar üçgen");  
        } else if (mA == mB || mB == mC || mA == mC) {  
            print("Geçerli bir ikizkenar üçgen");  
        } else {  
            print("Geçerli sıradan bir üçgen");  
        }  
    } else {  
        print("HATA: İç açılar toplamı 180 olmadığı için bu bir üçgen belirtmez!");  
    }  
}
```

60- Problem Çözme Etkinliği - 2 (Çözümlü)

Soru: Verilen üç kenar uzunluğundan bir üçgen elde edilip edilemeyeceğini kontrol eden kodu yazınız. (Üçgen Eşitsizliği Kuralları).

```
void main() {  
    var a = 5;  
    var b = 4;  
    var c = 3;  
  
    // Üçgen eşitsizliği kuralı: Herhangi bir kenar, diğer iki kenarın  
    // toplamından küçük, farkının mutlak değerinden büyük olmalıdır.  
    if ((a + b > c && (a - b).abs() < c) &&  
        (a + c > b && (a - c).abs() < b) &&  
        (b + c > a && (b - c).abs() < a)) {  
        print("Bu kenar uzunlukları ile bir üçgen ÇİZİLEBİLİR.");  
    } else {  
        print("HATA: Üçgen eşitsizliğine uymuyor, üçgen ÇİZİLEMEZ!");  
    }  
}
```

61- switch-case Yapıları

Program akışını tek bir değişkenin aldığı farklı sabit değerlere göre dallandırmak amacıyla kullanılır.

Sadece **eşitlik** durumlarını kontrol edebilir, büyüktür/küçüktür gibi sorgulamalar yapamaz.

Her **case** bloğunun sonuna **break** konulmalıdır. **break** unutulursa kod bir sonraki case bloğuna taşabilir (fall-through).

Koşulların hiçbiri eşleşmezse isteğe bağlı tanımlanan **default** bloğu çalışır.

```
void main() {  
    var gunNumarasi = 2;  
  
    switch (gunNumarasi) {  
        case 1: print("pazartesi"); break;  
        case 2: print("salı"); break;  
        case 3: print("çarşamba"); break;  
        default:  
            print("gün numarası 1-7 arasında olmalı");  
            break;  
    }  
}
```

62- switch-case Yapılarında break İle Gelişmiş Kullanım

Eğer bir **case** bloğunun içi boş bırakılırsa ve altına **break** yazılmazsa, kod bir sonraki dolu **case** bloğuna kadar düşmeye (ilerlemeyi sürdürmeye) devam eder.

Bu özellik, birden fazla durumun aynı çıktıyı veya sonucu üreteceği senaryolarda kod tasarrufu sağlar.

```
void main() {  
    var sonuc = "";  
    var gun = "salı";  
  
    switch (gun) {  
        case "pazartesi":  
        case "salı":  
        case "çarşamba":  
        case "perşembe":  
        case "cuma":  
            sonuc = "hafta içindeki";  
            break; // Hafta içi günlerin hepsi buraya düşer ve durur.  
        case "cumartesi":  
        case "pazar":  
            sonuc = "hafta sonundaki";  
            break;  
    }  
  
    print("$gun $sonuc günlerden biridir");  
}
```

63- for Döngüleri

Belirli sayıda tekrarlanacak işlemler için kullanılan standart döngü yapısıdır.

Döngü içinde bir **kontrol değişkeni** tanımlanır, her adımda bu değişken manipüle edilir (artırılır/azaltılır).

Döngü gövdesi çalışmadan önce her seferinde **koşul ifadesi** test edilir. Koşul **false** olduğu an döngü sonlanır.

```
import 'dart:math';

void main() {
  var i = 0;
  double sinus = 0;
  double radyan = 0;

  // 0'dan 90'a kadar (90 dahil) birer birer artan döngü
  for (i = 0; i <= 90; i++) {
    radyan = i * pi / 180; // Dereceyi radyana çevirme
    sinus = sin(radyan); // dart:math kütüphanesinden sin fonksiyonu
    print("${\u1d52: $sinus}"); // \u1d52 karakteri derece (°) simgesini basar
  }
}
```

64- for-in Döngüleri

Bir dizideki (List) veya koleksiyondaki tüm elemanları sırayla taramak için kullanılır.

Geleneksel **for** döngüsündeki gibi bir indeks kontrolü veya artırma işlemi yapılmasına gerek kalmaz.

Yapı, koleksiyondaki her elemanı sırayla döngü bloğunda tanımlanan geçici değişkene atar. Koleksiyonlar üzerinde tarama yapmak için oldukça pratiktir.

```
void main() {  
    var elementler = ["hidrojen", "helyum", "lityum"];  
  
    // for-in yapısı ile tarama  
    for (var el in elementler) {  
        print(el + " " + el.length.toString());  
    }  
  
    // Alternatif tek satırlık forEach yöntemi:  
    // elementler.forEach((el) => print(el + " " + el.length.toString()));  
}
```

65- while Döngüleri

Belirtilen bir koşul **doğru (true) olduğu sürece** durmaksızın yinelenen döngülerdir.

for döngülerinden farkı, kontrol değişkeninin artırılması veya manipülasyonu otomatik değildir; döngü bloğu içerisinde el ile yapılmalıdır.

Eğer koşulu sonlandıracak manipülasyon kodu unutulursa, program **sonsuz döngüye (infinite loop)** girer ve kilitlenir.

```
void main() {  
    var elementler = ["hidrojen", "helyum", "lityum"];  
  
    // Liste boş olmadığı sürece çalış  
    while (elementler.isNotEmpty) {  
        var el = elementler.first; // İlk elemanı al  
        print(el);  
        elementler.removeAt(0); // İlk elemanı sil (Böylece liste bir gün  
        boşalacak)  
    }  
}
```

66- Fonksiyonlar (Functions)

Sıklıkla uygulanan bir işlemi gerçekleştirmek adına bir araya getirilmiş, tekrar kullanılabilir kod bloklarıdır.

Fonksiyonlar dışarıdan **parametreler** alabilir ve işlem sonucunda geriye bir değer **döndürebilir (return)**.

Değer döndürmeyen fonksiyonların başına veri tipi yerine **void** yazılır.

Tek satırlık işlem barındıran fonksiyonlarda süslü parantez ve **return** yerine ok sentaksı (**=>**) kullanılabilir.

```
// Standart uzun tanımlama
bool ciftMi(int sayi) {
    return (sayi % 2 == 0) ? true : false;
}

// Ok (Arrow) sentaksı ile tek satırda kısa yazım alternatifi
// bool ciftMiKisa(int sayi) => (sayi % 2 == 0) ? true : false;

void main() {
    print(ciftMi(4)); // Çıktı: true
}
```

67- Fonksiyonlarda Parametre Türleri: Pozisyonel ve İsimli

Fonksiyonlar parametreleri iki farklı yöntemle kabul edebilir:

1. **Pozisyonel Parametreler (Positional):** Parametrelerin gönderilme sırası, fonksiyonun tanımındaki sıra ile birebir aynı olmak zorundadır.
2. **İsimli Parametreler (Named):** Parametreler süslü parantez `{ }` içine alınır. Çağrılırken sıra önemli değildir, değerler parametre isimleriyle eşleştirilerek gönderilir.

```
// Pozisyonel Parametre örneği
double alanHesapla(double taban, double yuksekli) {
    return (taban * yuksekli) / 2;
}

// İsimli Parametre örneği
double alanHesapla2({double taban = 0, double yukseklik = 0}) {
    return (taban * yukseklik) / 2;
}

void main() {
    print(alanHesapla(3, 4)); // Sıra zorunlu
    print(alanHesapla2(yukseklik: 4, taban: 5)); // Sıra önemsiz, isimler belirleyici
}
```

68- Anonim (İsimsiz) Fonksiyonlar (Anonymous Functions)

Belirli bir adı olmayan ve genellikle tek seferlik işlemler veya koleksiyon metotları (**forEach** vb.) içerisinde geçici olarak tanımlanan fonksiyonlardır.

Blok içerisinde birden fazla satır veya yönerge kullanılacaksa ok sentaksı (**=>**) yerine süslü parantezler **{ }** tercih edilmelidir.

```
void main() {  
    var sıra = 1;  
    var elementler = ["hidrojen", "helyum"];  
  
    // forEach içerisine anonim bir fonksiyon tanımlanmıştır  
    elementler.forEach((elementim) {  
        print("$elementim elementi $sıra. sıradadır");  
        sıra++; // Birden fazla yönerge olduğu için süslü parantez kullanıldı  
    });  
}
```

69- Etki Alanları (Lexical Scope)

Dart dilinde süslü parantezler `{ }` ile kapatılan her kod bloğu kendine ait bir etki alanı (**scope**) oluşturur.

Bir değişken, yalnızca **tanımlandığı etki alanı ve onun alt (iç) alanlarında** geçerlidir.

Üst veya dış etki alanından, alt alanda tanımlanmış yerel bir değişkene erişilemez; erişilmeye çalışıldığında derleme hatası (**Getter not found**) alınır.

```
void main() {  
  for (var i = 0; i < 5; i++) {  
    print("$i"); // i değişkeni sadece for bloğu içinde yaşar  
  }  
  // HATA: i değişkenine bu dış alandan erişilemez!  
  // print("$i");  
}
```

Her fonksiyonun kendi etki alanı bağımsızdır. Fonksiyonlar arası veri aktarımı parametreler ve **return** mekanizması ile sağlanır.

Dart'ta fonksiyonlara parametre gönderildiğinde, değişkendeki **değer kopyalanarak** gönderilir (İlkel tipler için). Dolayısıyla alt fonksiyonda parametre üzerinde yapılan değişiklik ana fonksiyondaki orijinal değişkeni etkilemez.

```
void main() {  
    var fiyat = 100;  
    var kdv = 18;  
    var sonuc = ekle(fiyat, kdv);  
  
    print("Dönen sonuç: $sonuc"); // 128  
    print("Orijinal fiyat değeri değişmedi: $fiyat"); // 100  
}  
  
int ekle(int sayi1, int sayi2) {  
    sayi1 += 10; // Yerel kopyada değişiklik yapılır  
    var toplam = sayi1 + sayi2;  
    return toplam;  
}
```

Bir iç etki alanı (alt fonksiyon veya blok), kendi üstünde yer alan dış etki alanındaki değişkenlere doğrudan erişebilir ve içeriklerini kalıcı olarak değiştirebilir.

```
void main() {  
    var fiyat = 100;  
    var kdv = 18;  
    var toplam = 0;  
  
    int hesapla() {  
        var sonuc = fiyat + kdv;  
        var otv = 6;  
        toplam = fiyat + kdv + otv; // Dışarıdaki 'toplam' değişkenini kalıcı olarak günceller  
        return sonuc;  
    }  
  
    print("Hesapla dönen: ${hesapla()}");  
    print("Güncellenmiş toplam değeri: $toplam"); // 124 yazar  
}
```

Dart, nesneye yönelik bir programlama dilidir. Nesneler, birbirleriyle ilişkili veri yapılarını ve kod bloklarını bir arada tutan paketlerdir.

Nesnelerin dış dünyaya sunduğu iki temel bileşen vardır:

- **Özellikler (Properties / Attributes):** Nesnenin tuttuğu veriler (Örn: `liste.length`).
- **Metotlar (Methods):** Nesne üzerinde işlem yapan fonksiyonlar (Örn: `liste.add()`).

Nesne referanslarının arkasına getirilen nokta (`.`) işleci ile bu özellik ve metotlara erişilir.

```
void main() {  
    var sayilar = [1, 2, 3, 5, 8, 13]; // Bir liste nesnesi üretildi  
    sayilar.add(21); // add() metodu kullanıldı  
    var elemanSayisi = sayilar.length; // length özelliği okundu  
}
```

- Sınıf (Class): Nesnelerin şablonu veya taslağıdır. Nesnelerde bulunacak özellikler ve metotlar sınıflar içinde kodlanır.
- Enum (Enumeration): Önceden tanımlanmış, sabit ve değişmez değerler kümesini ifade eden veri tipleridir (Örn: Cinsiyetler, renkler, haftanın günleri). Kodda standartlaşma ve okunurluk sağlar.

74- Kod Örneği - Sınıf, Enum, Getter/Setter ve Constructor Yapısı

Derste tahtada veya VS Code üzerinde doğrudan çalıştırabileceğin tam **Hayvan** sınıfı örneği:

```
// Sınıf dışında tanımlanmış Enum yapısı
enum Cinsiyetler { erkek, disi }

class Hayvan {
    int _yas; // Önündeki '_' işareti bu değişkeni private (gizli) yapar.
    Cinsiyetler cinsiyet;

    // Yapıcı Metot (Constructor) - Kısa sentaks kullanımı
    Hayvan(this._yas, this.cinsiyet);

    // Yaşı düzenleyen Setter metodu
    set yasBelirle(int yeniYas) {
        if (yeniYas >= 0) {
            _yas = yeniYas;
        }
    }

    // Yaşı geri döndüren Getter metodu
    int get yasOgren => _yas;
}

void main() {
    // Nesne oluşturma (Constructor çağırısı)
    var kus = Hayvan(2, Cinsiyetler.disi);

    kus.yasBelirle = 3; // Setter tetiklendi
    print("Kuşun Yaşı: ${kus.yasOgren}"); // Getter tetiklendi
    print("Kuşun Cinsiyeti: ${kus.cinsiyet}");
}
```

75- İsimli Yapıcı Metotlar (Named Constructors)

Bir sınıfta aynı isimle birden fazla geleneksel yapıcı metot (constructor) tanımlanamaz.

Dart, bu sınırlamayı aşmak için **İsimli Yapıcı Metotlar** (**`SinifAdi.metotAdi`**) yapısını sunar.

Çok sayıda özelliğe (property) sahip sınıflarda, pozisyonel parametreler yerine isimli yapıcı metotlar kullanmak kod okunurluğunu ve esnekliği artırır.

Zorunlu kılınması gereken isimli parametrelerin başına **required** kelimesi getirilir.

İstenirse bazı parametrelere varsayılan (default) değerler atanabilir.

Yapıcı metodun gövdesi çalıştırılmadan önce değişkenlere ilk değer ataması yapmak için **Başlatıcı Listeler (:)** kullanılır.

Sınıf isminin ve parametrelerinin bittiği yere koyulan : işaretinden sonra atama çiftleri aralarına virgül koyularak sıralanır.

Bu alan, özellikle **final** değişkenlere ilk değer atamak veya nesne yaratılırken parametreleri doğrulamak için idealdir.

```
// hayvan.dart
enum cinsiyetler { disi, erkek }

class Hayvan {
  int _yas;
  cinsiyetler cinsiyet;

  // 1. Standart Yapıcı Metot (Pozisyonel)
  Hayvan(this._yas, this.cinsiyet);

  // 2. İsimli Yapıcı Metot + Başlatıcı Liste (Initializer List) + Varsayılan Değer
  Hayvan.isimli({required int yas, this.cinsiyet = cinsiyetler.erkek}) : _yas = yas;

  // Setter ve Getter mekanizmaları
  set yasBelirle(int yas) => _yas = yas;
  int get yasOgren => _yas;

  void besle() => print("yedim");

  void raporla() {
    print("$_yas yaşımdayım, cinsiyetim: $cinsiyet");
  }
}
```

```
// main.dart
import './hayvan.dart';

void main() {
  // Standart constructor ile nesne üretimi
  var kus = Hayvan(2, cinsiyetler.disi);
  kus.besle();
  kus.raporla();

  // İsimli constructor ile nesne üretimi (cinsiyet verilmedi, erkek varsayıldı)
  var kedi = Hayvan.isimli(yas: 5);
  kedi.yasBelirle = 3;

  print(kedi.cinsiyet); // Çıktı: cinsiyetler.erkek
  print(kedi.yasOgren); // Çıktı: 3
  kedi.raporla();
}
```

79- Kalıtım (Inheritance)

Nesneye yönelik programlamanın en güçlü mekanizmalarından biridir.

Bir sınıfın (alt sınıf/subclass), başka bir sınıfın (üst sınıf/superclass) özellik ve metotlarını miras almasını sağlar.

Kalıtım ilişkisi kurmak için **extends** anahtar sözcüğü kullanılır.

Alt sınıf nesnesi oluşturulurken, üst sınıfın yapıcı metodunu tetiklemek için başlatıcı listesinde **super()** çağrısı yapılır.

80- Metot Ezme (Method Overriding)

Üst sınıftan miras alınan bir metodun, alt sınıfın ihtiyaçlarına göre yeniden şekillendirilip/değiştirilip yazılması işlemine **Metot Ezme (Override)** denir.

Kodun okunabilirliğini artırmak ve ezme işlemini açıkça belirtmek için metodun üzerine **@override** ibaresi eklenir.

Eğer alt sınıftaki yeni metodun içinde üst sınıftaki orijinal metodun da çalışması isteniyorsa **super.metotAdi()** ifadesi kullanılır.

81- Kod Örneği - Kalıtım Uygulaması (kedi.dart)

```
// kedi.dart
import './hayvan.dart';

enum cins { van_kedisi, ankara_kedisi, tekir }

// Kedi sınıfı Hayvan sınıfından türetilmiştir
class Kedi extends Hayvan {
  cins cinsi;

  // super() çağırısı ile gelen parametreler üst sınıfa (Hayvan) paslanır
  Kedi(this.cinsi, int yas, cinsiyetler cinsiyet) : super(yas, cinsiyet);

  void sesCikar() => print("miyav");

  @override
  void raporla() {
    super.raporla(); // Üst sınıftaki raporla() metodu çalışır
    print("cinsim: $cinsi"); // Ekstra kediye özgü çıktı eklenir
  }
}
```

82- Kod Örneği - Türetilmiş Sınıfın Kullanımı (main.dart)

```
// main.dart (Kalıtım Sonrası)
import './kedi.dart';
import './hayvan.dart';

void main() {
  // Kedi nesnesi oluşturuluyor
  var kedim = Kedi(cins.ankara_kedisi, 6, cinsiyetler.erkek);

  // Hem ezilen raporla() hem de kediye özgü sesCikar() çağrılıyor
  kedim.raporla();
  kedim.sesCikar();
}
```

Soru: En az 3 farklı hayvan sınıfı (Ör: Memeliler, Sürüngenler, Kuşlar) ve her sınıfa ait bir tür içeren hiyerarşiyi kalıtım kullanarak tasarlayınız.

```
// hiyerarşi tasarımı örnek kodu
class Canli {
    void nefesAl() => print("Nefes alındı.");
}

class Memeli extends Canli {
    void sutBesle() => print("Yavrular sütle beslendi.");
}

class Kus extends Canli {
    void uc() => print("Kuş uçuyor.");
}

class EvKedisi extends Memeli {
    void miyavla() => print("Miyav!");
}

void main() {
    var kedi = EvKedisi();
    kedi.nefesAl(); // Canli sınıfından miras
    kedi.sutBesle(); // Memeli sınıfından miras
    kedi.miyavla(); // Kendi metodu
}
```

84- İlave Etkinlik - Asal Sayı Kontrolü

Bir sayının asal olup olmadığını test eden algoritma örneğidir. Kitaptaki `sqrt()` (karekök) kullanımı için `dart:math` kütüphanesine ihtiyaç duyulmaktadır.

```
import 'dart:math';

void main() {
  var sayi = 7;
  var sayac = 2;
  var bulundu = false;

  // Sayının kareköküne kadar olan sayılara bölünme kontrolü performansı artırır
  while (sayac <= sqrt(sayi)) {
    if (sayi % sayac == 0) {
      bulundu = true;
      break; // Bölünen ilk sayıda döngüden çıkılır
    }
    sayac++;
  }

  if (bulundu) {
    print("${sayi.toString()} sayısı asal değildir");
  } else {
    print("${sayi.toString()} sayısı asaldır");
  }
}
```

<https://create.kahoot.it/share/mobil-uygulama-2-ders/158f2eb8-07be-403d-a714-bb59ff784eb4>

Muhammed Emin Korkunç